THE THREE-VARIABLE CONFLICT-CLAUSE POSET: GRADED STRUCTURE, MODEL COUNTING BY UNOBSTRUCTED ATOMS, AND STEP-COUNTER BOUNDS, FORMALIZED IN LEAN 4

BENJAMIN STANLEY FROHMAN

Abstract. We study the finite poset L_3 of all 27 clauses over three Boolean variables, ordered by literal inclusion (equivalently, by logical implication $c_1 \models c_2$ for sub-clauses). We prove in Lean 4 with Mathlib that L_3 is graded with layer sizes $1, 6, 12, 8$ and 54 covering edges; that literal inclusion is a partial order with least element the empty clause and with a componentwise meet that is a greatest lower bound; that the alternating layer sum is -1; and that for every CNF formula Φ over the three variables the number of satisfying assignments equals the number of rank-3 clauses not implied by any clause of Φ. We also formalize a step-counting state monad and prove exact tick counts and upper bounds for two explicitly defined simulation programs, including a bound of the form $(n+m)^2 + 2^{n/(\log n)^{1/2+\varepsilon}}$ for the block program run at block size $\lceil (\log n)^{1/2+\varepsilon} \rceil$. These bounds concern the tick counters of the defined programs only; no claim is made about the complexity of SAT, #SAT, or the Exponential Time Hypothesis. All proofs are free of `sorry` and `native_decide` and use only Lean's standard axioms; the main results are bundled in a single master theorem.

1. Introduction

Clause learning in SAT solvers manipulates clauses ordered by subsumption. For a fixed small number of variables this order is a finite poset that can be analysed exhaustively. This paper gives a complete, kernel-checked account of the case of three variables, together with an exact identity relating model counting to the poset, and a small formal framework for counting the steps of programs written in a state monad.

We emphasize at the outset what is *not* claimed. The step bounds of Section 5 are statements about two specific programs whose only effect is to increment a counter by prescribed amounts (for instance, by n^2 and then by 2^n). They are exact by construction and do not measure the running time of any SAT algorithm. Nothing in this paper bears on P vs. NP or on the Exponential Time Hypothesis.

2. The poset L_3

A *variable state* is an element of $\{\bot, \top, \mathsf{F}\}$: the variable is absent, appears positively, or appears negatively. A *clause* over x_1, x_2, x_3 (Lean: `ValidClause3`) is a triple of variable states; there are $3^3 = 27$ of them (Lean: `allValidClauses3`). The *rank* of a clause is its number of literals. For variable states, $s_1 \sqsubseteq s_2$ iff $s_1 = \bot$ or $s_1 = s_2$; for clauses, $c_1 \preceq c_2$ iff this holds componentwise (Lean: `ImpliesPoset`), i.e. the literals of c_1 are among those of c_2, so $c_1 \models c_2$. A covering edge is a pair $c_1 \preceq c_2$ with $\operatorname{rank} c_2 = \operatorname{rank} c_1 + 1$. The meet $c_1 \wedge c_2$ keeps, in each coordinate, a literal common to both and is $\bot$ otherwise (Lean: `clauseMeet`).

Theorem 2.1 (Graded structure)**.** (1) $|L_3| = 27$*; the layers of rank* $0, 1, 2, 3$ *have sizes* $1, 6, 12, 8$*, and they partition* L_3*.*

(2) *There are exactly* 54 *covering edges:* 6 *from rank* 0*,* 24 *from rank* 1*, and* 24 *from rank* 2*.*

Theorem 2.2 (Order and meet)**.** *For all clauses* c, c_1, c_2, c_3*:*

Date: November 4, 2026.

(1) $c \preceq c$; *if* $c_1 \preceq c_2$ *and* $c_2 \preceq c_1$ *then* $c_1 = c_2$; *if* $c_1 \preceq c_2 \preceq c_3$ *then* $c_1 \preceq c_3$;
(2) $\emptyset \preceq c$, *where* $\emptyset$ *is the empty clause;*
(3) $c_1 \wedge c_2 \preceq c_1$ *and* $c_1 \wedge c_2 \preceq c_2$, *and* $c \preceq c_1 \wedge c_2$ *iff* $c \preceq c_1$ *and* $c \preceq c_2$.

The corresponding statements for single variable states (reflexivity, antisymmetry, transitivity of $\sqsubseteq$; $s \sqsubseteq s_1 \wedge s_2 \iff s \sqsubseteq s_1 \wedge s \sqsubseteq s_2$; the meet lies below both arguments) and the componentwise characterization of $\preceq$ are proved as lemmas.

3. Alternating sums and boundary identities

Proposition 3.1. $|L^{(0)}| - |L^{(1)}| + |L^{(2)}| - |L^{(3)}| = 1 - 6 + 12 - 8 = -1$ *and* $|L^{(1)}| - |L^{(2)}| + |L^{(3)}| = 2$, *where* $L^{(k)}$ *is the rank-*k *layer.*

For $f : L_3 \to \mathbb{Z}$ and an ordered pair (v_0, v_1) put $\partial f(v_0, v_1) = f(v_1) - f(v_0)$, and for a triple (v_0, v_1, v_2) put $\partial^2 f(v_0, v_1, v_2) = \partial f(v_1, v_2) - \partial f(v_0, v_2) + \partial f(v_0, v_1)$. Similarly, for g defined on ordered pairs, put $\delta g(v_0, v_1, v_2) = g(v_1, v_2) - g(v_0, v_2) + g(v_0, v_1)$ and let $\delta^2 g(v_0, \ldots, v_3)$ be the alternating sum of δg over the four faces.

Proposition 3.2. *For all* $f : L_3 \to \mathbb{Z}$, *all* $g : L_3 \times L_3 \to \mathbb{Z}$ *and all* $v_0, \ldots, v_3 \in L_3$: $\partial^2 f(v_0, v_1, v_2) = 0$ *and* $\delta^2 g(v_0, v_1, v_2, v_3) = 0$.

Remark 3.3. These are the identities $\partial \circ \partial = 0$ evaluated against cochains. They hold for *arbitrary* tuples of clauses, not only for chains of L_3, and their proofs are linear-arithmetic cancellation; they use no property of the order. The formalization does not define the order complex of L_3 or compute its homology.

4. Model counting by unobstructed atoms

An assignment is $\alpha \in \{0, 1\}^3$. A clause is satisfied by α if one of its literals is true under α (the empty clause is never satisfied), and a CNF formula, given as a list Φ of clauses, is satisfied if all its clauses are. Let $\#\Phi$ be the number of satisfying assignments (Lean: `modelCount3`). For α, its *dual atom* A_α is the rank-3 clause with literal $\neg\alpha_i$ on each x_i, the unique full clause falsified by α. A rank-3 clause A is *unobstructed* by Φ if no $c \in \Phi$ satisfies $c \preceq A$.

Lemma 4.1. (1) *For every clause* c *and assignment* α: α *satisfies* c *iff* $c \not\preceq A_\alpha$.
(2) *Hence* α *satisfies* Φ *iff* A_α *is unobstructed by* Φ.
(3) *The map* $\alpha \mapsto A_\alpha$, *applied to the eight assignments in their standard enumeration, lists exactly the rank-*3 *layer.*

Theorem 4.2 (`modelCount3_eq_unobstructedAtomCount3`)**.** *For every CNF formula* Φ *over* x_1, x_2, x_3, $\#\Phi$ *equals the number of rank-*3 *clauses unobstructed by* Φ.

5. Step counting

A computation in the state monad CountM carries a natural-number counter; tick(k) adds k. Two programs are defined:

- simLattice(n): tick($n \cdot n$); tick(2^n); return true.
- simBlock(n, m, k): tick($(n+m)^2$); tick($2^{\lfloor n/\max(k,1) \rfloor}$); return true.

Let $\ell(n, \varepsilon) = (\log n)^{1/2+\varepsilon}$ (natural logarithm, real power) and $b(n, \varepsilon) = \lceil \ell(n, \varepsilon) \rceil$.

Theorem 5.1 (Tick counts)**.** (1) *From counter* s, tick(k) *ends at* $s + k$; tick(a); tick(b) *ends at* $s + a + b$.
(2) *From* s, simLattice(n) *returns true with counter* $s + n \cdot n + 2^n$, *and* simBlock(n, m, k) *returns true with counter* $s + (n+m)^2 + 2^{\lfloor n/\max(k,1) \rfloor}$.
(3) *Started from* 0, simLattice(n) *ends with exactly* $n^2 + 2^n$ *ticks (in particular at most that many), and* simBlock(n, m, k) *ends with at most* $(n+m)^2 + 2^{\lfloor n/\max(k,1) \rfloor}$ *ticks.*

Theorem 5.2 (Real-logarithmic bounds)**.** (1) *If $n \geq 2$ and $k \geq 2$ are natural numbers, then $\lfloor n/k \rfloor < n$.*

(2) *If $n \geq 2$, $k \geq 1$ and $\ell(n, \varepsilon) \leq k$, then the final count of* simBlock(n, m, k) *from* 0 *is at most $(n+m)^2 + 2^{n/\ell(n,\varepsilon)}$.*

(3) *In particular, for $n \geq 2$ and every real ε, running* simBlock$(n, m, b(n, \varepsilon))$ *from* 0 *ends with at most $(n+m)^2 + 2^{n/(\log n)^{1/2+\varepsilon}}$ ticks.*

(4) *For $n \geq 3$ and $\varepsilon \geq 0$: $2^{n/\ell(n,\varepsilon)} < 2^n$, and the final count $n^2 + 2^n$ of* simLattice(n) *strictly exceeds $n^2 + 2^{n/\ell(n,\varepsilon)}$.*

Supporting facts proved along the way: $e < 3$; $\log n > 0$ for $n \geq 2$ and $\log n > 1$ for $n \geq 3$; $\ell(n, \varepsilon) > 0$ for $n \geq 2$ and $\ell(n, \varepsilon) > 1$ for $n \geq 3, \varepsilon \geq 0$; $\ell \leq b$; $b \geq 1$ for $n \geq 2$ and $b \geq 2$ for $n \geq 3, \varepsilon \geq 0$.

Proposition 5.3 (`denominator_divergence`)**.** *If $o : \mathbb{R} \to \mathbb{R}$ satisfies $o = o(1)$ as $x \to \infty$, then $(\log x)^{1/2+o(x)} \to \infty$ as $x \to \infty$.*

Remark 5.4. Theorems 5.1 and 5.2 are exact statements about the counters of the two programs above, whose tick amounts are written into their definitions. They are not lower or upper bounds on any algorithm for satisfiability or model counting.

6. Master theorem

Theorem 6.1 (`eth_conflict_clause_lattice_master_product_theorem`)**.** *The conjunction of: Theorem 2.1 (the counts $27, 1, 6, 12, 8, 54$); Theorem 2.2(1),(2) and the meet characterization in (3); Proposition 3.2 and the first identity of Proposition 3.1; Theorem 4.2; the bounds of Theorem 5.1(3); and Theorem 5.2(1),(3) and the first inequality of (4).*

7. Correspondence between claims and Lean declarations

All names are in namespace `ConflictClauseLattice`; Lean 4 v4.30.0-rc1 with Mathlib.

Claim	Lean declaration(s)
Thm. 2.1(1)	`card_allValidClauses3`, `layer0_count`, `layer1_count`, `layer2_count`, `layer3_count`, `layer_partition_sum_three`
Thm. 2.1(2)	`coveringEdges3_count`, `coveringEdges_by_layer`
Thm. 2.2(1)	`impliesPoset_refl`, `impliesPoset_antisymm`, `impliesPoset_trans`
Thm. 2.2(2)	`emptyClause3_minimal`
Thm. 2.2(3)	`clauseMeet_implies_left`, `clauseMeet_implies_right`, `implies_clauseMeet_iff`
Variable-state lemmas	`varStateSub_refl`, `varStateSub_antisymm`, `varStateSub_trans`, `varStateSub_meet_eq`, `varStateMeet_sub_left`, `varStateMeet_sub_right`, `impliesPoset_iff`
Prop. 3.1	`euler_poincare_L3`, `non_empty_euler_characteristic_L3`
Prop. 3.2	`boundary1_comp_boundary2_eq_zero`, `boundary2_comp_boundary3_eq_zero`
Lemma 4.1	`varStateSat_eq_not_varStateSub`, `satisfiesClause_iff_not_implies_dualAtom`, `satisfiesFormula_eq_isUnobstructed_dualAtom`, `dualAtoms_eq_layer3`
Thm. 4.2	`modelCount3_eq_unobstructedAtomCount3`

Claim	Lean declaration(s)
Thm. 5.1(1)	`tick_run`, `monad_reduction_two_ticks`
Thm. 5.1(2)	`simulateLatticeTraversal_run`, `simulateBlockHomologicalTraversal_run`
Thm. 5.1(3)	`state_injection_steps`, `lattice_traversal_steps_exact`, `lattice_traversal_complexity_bound`, `block_homological_traversal_bound`
Thm. 5.2(1)	`subexponential_exponent_strictly_smaller`
Thm. 5.2(2)	`block_homological_traversal_real_log_bound`
Thm. 5.2(3)	`simulateBlockHomologicalTraversal_at_logBlockSize_bound`
Thm. 5.2(4)	`subExponential_rpow_lt_two_pow`, `simulateLatticeTraversal_exceeds_subExponential_bound`
Supporting facts	`exp_one_lt_three`, `log_nat_pos_of_ge_two`, `one_lt_log_nat_of_ge_three`, `logDenominator_pos`, `logDenominator_gt_one`, `logDenominator_le_logBlockSize`, `logBlockSize_ge_one`, `logBlockSize_ge_two`
Prop. 5.3	`denominator_divergence`
Thm. 6.1	`eth_conflict_clause_lattice_master_product_theorem`

Appendix A. Verification certificate

The formal development was machine-checked as follows.

- Repository:
  `https://github.com/BenFrohman/eth-conflict-clause-lattice`
- Verified commit:
  `8b03b20f6393c3c4dbc419b02932308fa99c116c`
- Lean toolchain: `leanprover/lean4:v4.30.0-rc1`
- Mathlib: tag `v4.30.0-rc1`, commit (from `lake-manifest.json`)
  `0c154d67103f74be3a0f2c509f72ccbf5be9f2a7`
- Continuous integration run (build, axiom audit, `sorry`/`axiom` scan; passed):
  `https://github.com/BenFrohman/eth-conflict-clause-lattice/actions/runs/41794839338`
- SHA-256 of the GitHub source tarball of the verified commit, downloaded twice with identical result:
  `https://github.com/BenFrohman/eth-conflict-clause-lattice/archive/8b03b20f6393c3c4dbc419b02932308fa99c116c.tar.gz`
  `fd949b7876a66601ae0b5dc7a67cea9814c4823d62e84fad4a4dc934e4135798`

The tarball hash refers to the verified commit rather than to a release tag, since a document cannot contain the hash of an archive that includes itself. The Lean sources are unchanged between that commit and the release tag.

The development contains no `sorry` and no user-declared axioms. Every one of the 54 theorems and lemmas listed below depends only on Lean's standard axioms `propext`, `Classical.choice` and `Quot.sound` (or on none). The table is the verbatim output of `lake env lean AxiomAudit.lean` (`#print axioms`) on the verified commit.

Lean declaration	`#print axioms`
`ConflictClauseLattice.eth_conflict_clause_lattice_master_product_theorem`	`[propext, Classical.choice, Quot.sound]`

Lean declaration	#print axioms
ConflictClauseLattice.boundary1_comp_boundary2_eq_zero	[propext, Quot.sound]
ConflictClauseLattice.boundary2_comp_boundary3_eq_zero	[propext, Quot.sound]
ConflictClauseLattice.dualAtoms_eq_layer3	(no axioms)
ConflictClauseLattice.euler_poincare_L3	(no axioms)
ConflictClauseLattice.modelCount3_eq_unobstructedAtomCount3	[propext]
ConflictClauseLattice.non_empty_euler_characteristic_L3	(no axioms)
ConflictClauseLattice.satisfiesClause_iff_not_implies_dualAtom	(no axioms)
ConflictClauseLattice.satisfiesFormula_eq_isUnobstructed_dualAtom	(no axioms)
ConflictClauseLattice.varStateSat_eq_not_varStateSub	(no axioms)
ConflictClauseLattice.card_allValidClauses3	(no axioms)
ConflictClauseLattice.clauseMeet_implies_left	[propext]
ConflictClauseLattice.clauseMeet_implies_right	[propext]
ConflictClauseLattice.coveringEdges3_count	(no axioms)
ConflictClauseLattice.coveringEdges_by_layer	(no axioms)
ConflictClauseLattice.emptyClause3_minimal	(no axioms)
ConflictClauseLattice.impliesPoset_antisymm	[propext]
ConflictClauseLattice.impliesPoset_iff	[propext]
ConflictClauseLattice.impliesPoset_refl	[propext]
ConflictClauseLattice.impliesPoset_trans	[propext]
ConflictClauseLattice.implies_clauseMeet_iff	[propext]
ConflictClauseLattice.layer0_count	(no axioms)
ConflictClauseLattice.layer1_count	(no axioms)
ConflictClauseLattice.layer2_count	(no axioms)
ConflictClauseLattice.layer3_count	(no axioms)
ConflictClauseLattice.layer_partition_sum_three	(no axioms)
ConflictClauseLattice.varStateMeet_sub_left	[propext]
ConflictClauseLattice.varStateMeet_sub_right	[propext]
ConflictClauseLattice.varStateSub_antisymm	(no axioms)
ConflictClauseLattice.varStateSub_meet_eq	[propext]
ConflictClauseLattice.varStateSub_refl	(no axioms)
ConflictClauseLattice.varStateSub_trans	(no axioms)
ConflictClauseLattice.block_homological_traversal_bound	[propext, Quot.sound]
ConflictClauseLattice.block_homological_traversal_real_log_bound	[propext, Classical.choice, Quot.sound]
ConflictClauseLattice.denominator_divergence	[propext, Classical.choice, Quot.sound]
ConflictClauseLattice.exp_one_lt_three	[propext, Classical.choice, Quot.sound]
ConflictClauseLattice.lattice_traversal_complexity_bound	[propext, Quot.sound]
ConflictClauseLattice.lattice_traversal_steps_exact	[propext, Quot.sound]
ConflictClauseLattice.logBlockSize_ge_one	[propext, Classical.choice, Quot.sound]
ConflictClauseLattice.logBlockSize_ge_two	[propext, Classical.choice, Quot.sound]
ConflictClauseLattice.logDenominator_gt_one	[propext, Classical.choice, Quot.sound]
ConflictClauseLattice.logDenominator_le_logBlockSize	[propext, Classical.choice, Quot.sound]
ConflictClauseLattice.logDenominator_pos	[propext, Classical.choice, Quot.sound]
ConflictClauseLattice.log_nat_pos_of_ge_two	[propext, Classical.choice, Quot.sound]

Lean declaration	`#print axioms`
`ConflictClauseLattice.monad_reduction_two_ticks`	`(no axioms)`
`ConflictClauseLattice.one_lt_log_nat_of_ge_three`	`[propext, Classical.choice, Quot.sound]`
`ConflictClauseLattice.simulateBlockHomologicalTraversal_at_logBlockSize_bound`	`[propext, Classical.choice, Quot.sound]`
`ConflictClauseLattice.simulateBlockHomologicalTraversal_run`	`[propext]`
`ConflictClauseLattice.simulateLatticeTraversal_exceeds_subExponential_bound`	`[propext, Classical.choice, Quot.sound]`
`ConflictClauseLattice.simulateLatticeTraversal_run`	`[propext]`
`ConflictClauseLattice.state_injection_steps`	`[propext]`
`ConflictClauseLattice.subExponential_rpow_lt_two_pow`	`[propext, Classical.choice, Quot.sound]`
`ConflictClauseLattice.subexponential_exponent_strictly_smaller`	`[propext, Quot.sound]`
`ConflictClauseLattice.tick_run`	`(no axioms)`

References

[1] R. Impagliazzo and R. Paturi, *On the complexity of k-SAT*, J. Comput. System Sci. **62** (2001), 367–375.

[2] J. P. Marques-Silva and K. A. Sakallah, *GRASP: a search algorithm for propositional satisfiability*, IEEE Trans. Comput. **48** (1999), 506–521.

[3] The mathlib Community, *The Lean mathematical library*, CPP 2020, 367–381.

Email address: `frohmanbenjamin@gmail.com`